Which Exploited Vulnerabilities Become Ransomware Targets?

A machine learning analysis of the CISA Known Exploited Vulnerabilities catalog joined to MITRE CVE Program records

Name ……

Student / index number ……

Module ……

Date of submission ……

Repository link (must be publicly accessible)
GitHub or Google Drive URL ……

1. Introduction

1.1 Project objective

This project analyses vulnerabilities that are confirmed to have been exploited by real attackers, and asks two questions of them. First, how quickly is a disclosed vulnerability weaponised, and can that speed be predicted from the vulnerability's own technical characteristics? Second, which exploited vulnerabilities go on to be adopted into ransomware campaigns?

The first question produces a negative result that turned out to be the most instructive part of the work. The second produces a usable classifier. Both are reported, because a project that only presents the analysis that worked would hide the more interesting finding.

1.2 Dataset description

The dataset was constructed for this project rather than downloaded ready-made. Two authoritative sources were joined.

The CISA Known Exploited Vulnerabilities catalog is maintained by the US Cybersecurity and Infrastructure Security Agency. A vulnerability is added only when CISA holds reliable evidence of active exploitation, and US federal agencies are then legally obliged to remediate it within a set deadline. This makes the catalog a ground-truth label for real-world danger, as distinct from CVSS, which records an analyst's assessment of how bad a vulnerability could be in principle.

The MITRE CVE Program record list holds the authoritative record for every published CVE. A build script fetched all 1,694 records matching the KEV catalog and extracted the CVSS v3.1 base score and its eight vector components, CISA's SSVC decision points (Exploitation, Automatable and Technical Impact), CWE weakness identifiers, publication dates, reference counts and affected-product counts.

Table 1. Dataset at a glance

Property	Value
Records	1,694 confirmed exploited vulnerabilities
Variables	39 (numerical, categorical, temporal and text)
Distinct vendors	283
Distinct CWE weakness types	249
CVE publication dates	2002 to 2026
KEV listing dates	2021-11-03 to 2026-09-02
Target A (regression)	exploitation_lag_days, continuous
Target B (classification)	is_ransomware, 20.8% positive (352 of 1,694)
Construction	build_dataset.py, included with this submission

The dataset satisfies the brief: well over six columns, a mixture of numerical and categorical variables, and enough records for model development. It also has genuine missing data, since it was assembled from live sources rather than curated, which makes the cleaning stage substantive.

1.3 Dataset source and access

Item	Reference
Source A	CISA Known Exploited Vulnerabilities catalog
	https://github.com/cisagov/kev-data
CISA KEV portal	https://www.cisa.gov/known-exploited-vulnerabilities-catalog
Source B	MITRE CVE Program, CVE List V5
	https://github.com/CVEProject/cvelistV5
CVE Program portal	https://www.cve.org
Data collected	2026-09-03

2. Methodology

All work was carried out in Python 3.12 using pandas, NumPy, scikit-learn, SciPy, matplotlib and seaborn. A fixed seed of 42 was used throughout and the package versions are pinned in requirements.txt, so results reproduce exactly.

2.1 Dataset construction

The build script reads the KEV catalog, derives the CVE Program record URL for each of the 1,694 CVE identifiers, and fetches them concurrently with retry handling. Each record is parsed for CVSS metrics, which may appear either in the CNA container supplied by the assigning organisation or in the CISA-ADP container added later, and the parser searches both. The extracted fields are joined back to the KEV catalog on the CVE identifier with a one-to-one validity check. Collection took 66 seconds and returned CVSS data for 1,632 records and CWE data for 1,478.

2.2 Data cleaning

Column names were converted from a mixture of camelCase and snake_case to snake_case throughout. Date fields were parsed and the two derived durations computed: exploitation lag as the gap between CVE publication and KEV listing, and remediation window as the gap between listing and CISA's deadline. No duplicate rows or duplicate CVE identifiers were present.

Missing data needed the most care. The eight CVSS vector components are absent as a block in 317 records, or 18.7%, and never partially. Testing showed the missingness is not random: a Mann-Whitney test on CVE year by vector availability returns p below 0.001, because CISA's enrichment programme has only recently begun back-filling older records. Deleting those rows would have silently removed the oldest and most historically significant vulnerabilities. Instead the categorical components were given an explicit NOT_ASSESSED level, a has_cvss_vector indicator was retained so a model can use the absence itself as a signal, CWE was given an UNKNOWN level on the same reasoning, and the 62 missing numeric base scores were imputed by the median within their severity band.

A negative exploitation lag means CISA confirmed exploitation before the CVE record was published. These 212 records are zero-days, not data errors, and were kept.

Vendor and CWE fields have high cardinality, 283 and 249 levels across 1,694 rows, so one-hot encoding either directly would have produced more columns than observations. Levels with fewer than 15 occurrences were collapsed into OTHER. Three count variables were winsorised at the 99th percentile rather than having rows removed, since a long tail in a count distribution is real.

2.3 Data preprocessing

The CVSS vector components are ordinal, not nominal: an attack vector of NETWORK is strictly more exposed than LOCAL, and PRIVILEGES_REQUIRED of NONE is strictly worse than HIGH. Encoding them as ordered integers preserves that structure, which one-hot encoding discards. The NOT_ASSESSED level was mapped to -1 so it remains distinguishable. SSVC fields were reduced to binary indicators.

Fields written by CISA at the moment of listing were excluded to prevent leakage: required_action, notes, due_date, remediation_window_days and the free-text description fields. The SSVC

Exploitation field was also dropped, since it takes the value 'active' for 1,692 of 1,694 records by construction. The final matrix holds 64 features. Data was split 80/20, stratified on the ransomware label for the classification task, and the scaler was fitted on the training split alone.

2.4 Data visualisation

Six figures were produced covering the distribution of exploitation lag, the growth and composition of the catalog, the relationship between CVE publication date and lag, severity against real-world outcome, the vendors and weakness types most represented, and the correlation structure of the engineered features. The third figure determined how the modelling section was structured.

2.5 Hypothesis testing

Three tests at alpha = 0.05, chosen to cover different families and each preceded by its assumption checks. Welch's t-test compares CVSS scores between ransomware-linked and other vulnerabilities. A chi-square test of independence examines SSVC Automatable against ransomware adoption. A Kruskal-Wallis test compares exploitation lag across attack vectors, chosen over one-way ANOVA because Shapiro-Wilk and Levene both reject the parametric assumptions. Effect sizes accompany every p-value, because with 1,694 records almost any real difference reaches significance.

2.6 Machine learning models

Two modelling tasks share the same feature matrix, which keeps their results comparable. Part A is a regression on log-transformed exploitation lag using linear regression, a random forest and gradient boosting, run twice: once on all records and once on the 817 CVEs published after the KEV catalog existed. Part B is a classification of ransomware adoption using logistic regression, a random forest tuned by grid search over nine parameter combinations, and gradient boosting, with balanced class weights throughout. Regression is scored by R squared and RMSE, classification by ROC-AUC, PR-AUC, precision, recall and F1, each with five-fold cross-validation.

3. Results and discussion

3.1 Findings from the visualisations

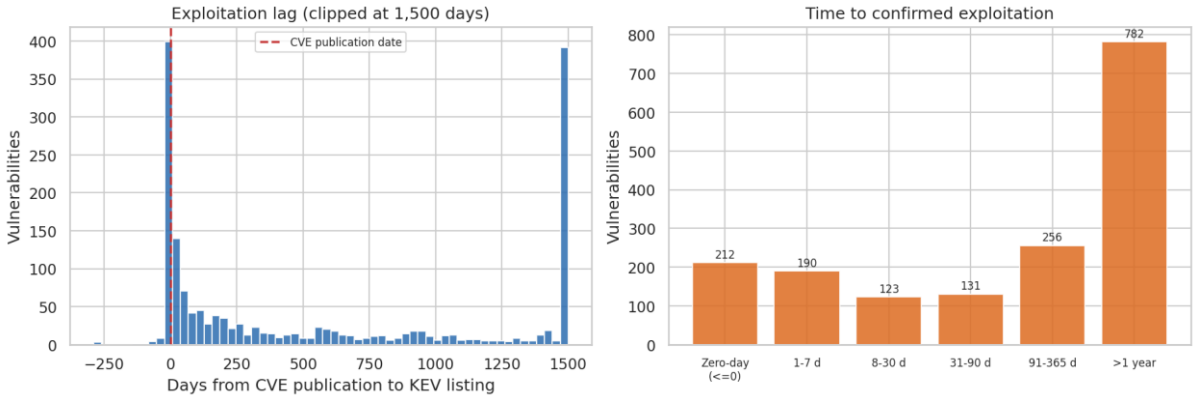


Figure 1. Time from CVE publication to confirmed exploitation.

The distribution is bimodal. 212 vulnerabilities were listed on or before their publication date, meaning exploitation was observed before public disclosure, and 402 within a week. At the other end, 782 took more than a year. The middle of the range is comparatively thin. That shape is the first indication that a single regression line will struggle, because these look like two different processes rather than one continuous one: vulnerabilities are either weaponised immediately as zero-days, or rediscovered and exploited long after the fact.

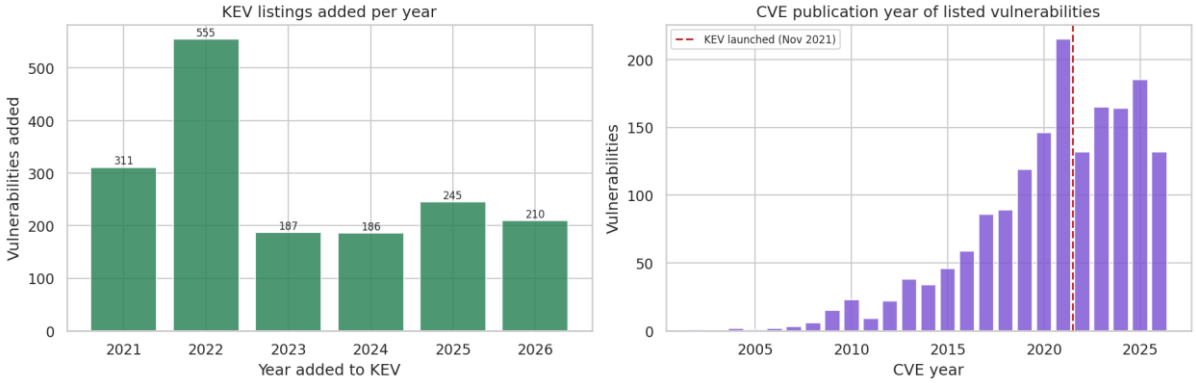


Figure 2. Catalog growth and the publication years it covers.

The catalog opened in November 2021 with a large initial back-fill, then settled into a steadier rate. The right panel shows the consequence that matters: the catalog contains CVEs published as far back as 2002, every one of which was necessarily added after November 2021.

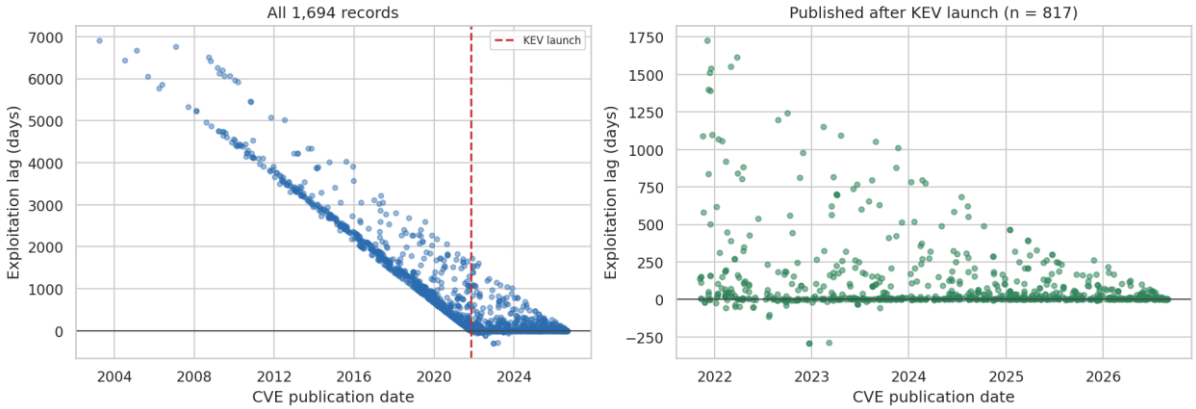


Figure 3. Exploitation lag against CVE publication date, before and after restriction.

This is the most important figure in the report, because the obvious way to model this data is wrong.

The left panel shows a hard diagonal ceiling. No vulnerability published in 2005 could have been listed before November 2021, so its recorded lag is at least sixteen years by arithmetic alone. The correlation between publication year and lag is -0.916, close to deterministic, and it reflects the date the catalog was created rather than anything about attacker behaviour. Any model trained on all 1,694 records will learn this boundary and report a good score for recovering a calendar constraint.

The right panel keeps only the 817 CVEs published after the catalog existed, where the correlation falls to -0.29. It does not fall to zero, and the faint remaining wedge shows why: a CVE published in mid-2026 cannot yet have a 1,000-day lag, because that much time has not passed. This is right-censoring rather than backlog, and it cannot be filtered away without discarding the most recent data. Section 4 returns to it.

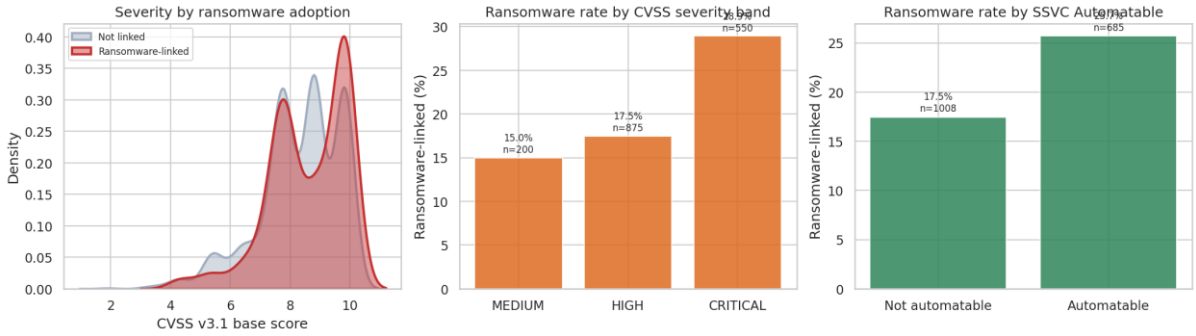


Figure 4. CVSS severity and SSVC Automatable against ransomware adoption.

CVSS barely separates the two groups. The density curves sit almost on top of each other and the ransomware rate climbs only slightly from the MEDIUM band to CRITICAL. SSVC's Automatable flag, which records whether an attacker can reliably script mass exploitation, separates them more cleanly: 25.7% against 17.5%. That is consistent with how ransomware operations actually select targets. They depend on reliable exploitation at volume, not on the worst-case impact of any single intrusion, so a merely severe vulnerability that is awkward to automate is of limited use to them.

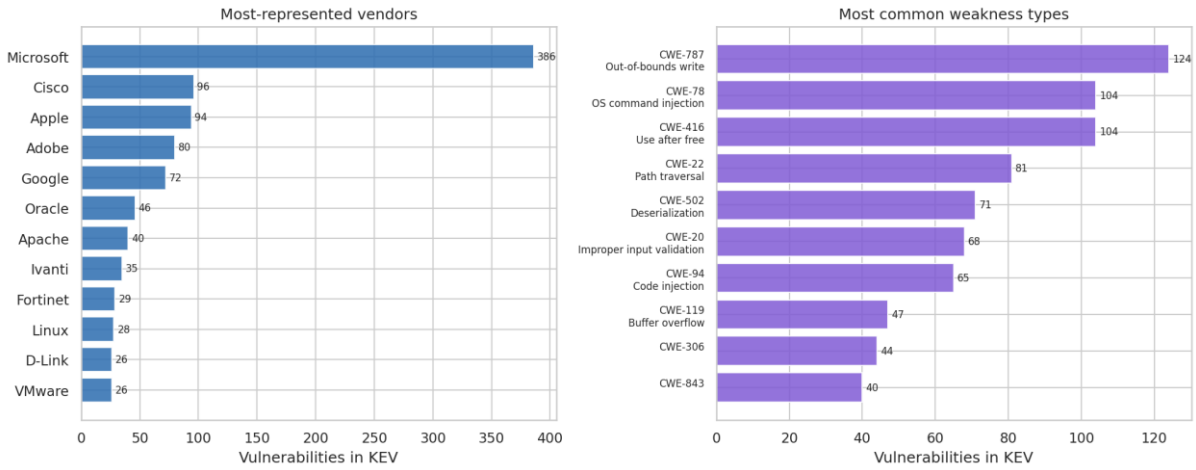


Figure 5. Most-represented vendors and weakness types in the catalog.

Microsoft accounts for 386 of the 1,694 entries, 22.8% of the catalog, and the top five vendors together account for a little under half. This reflects install-base size and monitoring intensity rather than a per-line defect rate, a point worth stating because the chart invites the opposite reading. Memory-safety weaknesses, out-of-bounds write and use after free, together with OS command injection and path traversal, dominate the weakness types, matching the pattern MITRE reports in its annual most-dangerous-weakness list.

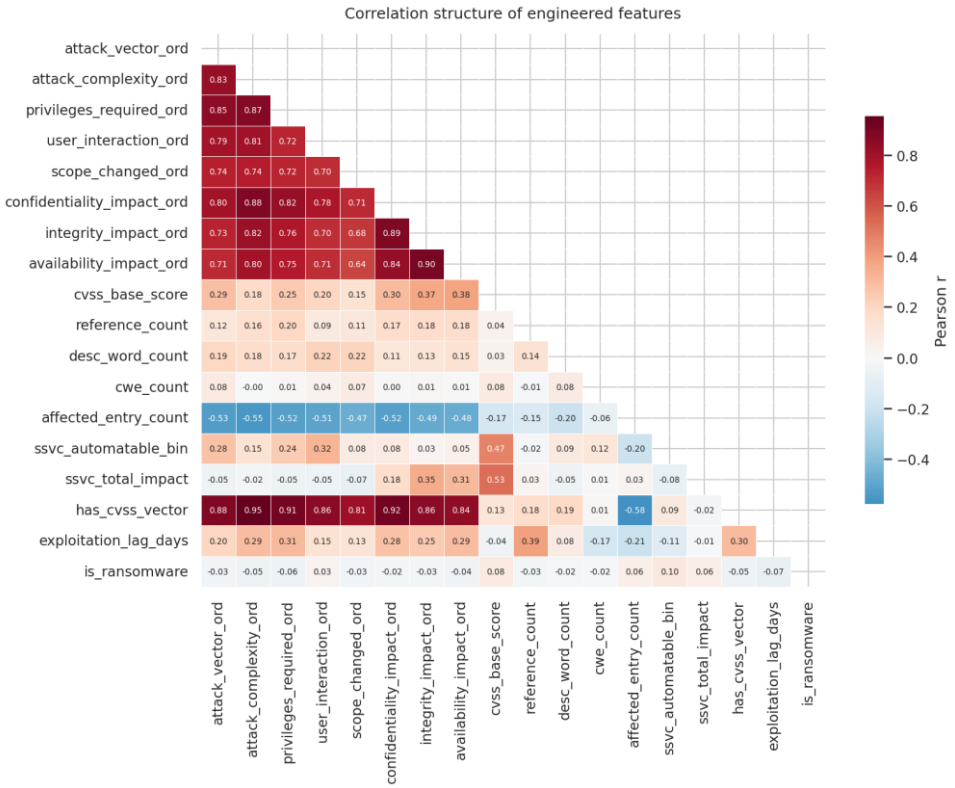


Figure 6. Correlation structure of the engineered features.

No single feature correlates strongly with ransomware adoption; the largest coefficients sit around 0.1 in absolute value. This says in advance that any classifier will have to combine many weak signals rather than exploit one dominant predictor, and it sets a realistic ceiling on the performance reported in Section 3.3.

3.2 Hypothesis testing results

Table 2. Summary of the three tests

Test	Statistic	p-value	Effect size	Decision
1. Welch t-test, CVSS by ransomware use	$t = 3.515$	0.00048	$d = 0.206$	Reject H_0
2. Chi-square, SSVC Automatable	$\chi^2 = 16.29$	5.4×10^{-5}	$V = 0.098$	Reject H_0
3. Kruskal-Wallis, lag by attack vector	$H = 5.533$	0.0629	$\varepsilon^2 = 0.0065$	Do not reject

Test 1: does CVSS severity distinguish ransomware targets?

H_0: mean CVSS base score is equal in both groups. H_1: the means differ. $\alpha = 0.05$.

Levene's test was run first to choose between the pooled and Welch forms. The result is $t = 3.515$ with $p = 0.00048$, so H_0 is rejected. Ransomware-linked vulnerabilities do score higher, averaging 8.577 against 8.302, a difference of 0.275 points on a ten-point scale.

The effect size is the part that matters. Cohen's d of 0.206 is small by any conventional benchmark, and a Mann-Whitney check agrees with the parametric test. With 1,694 observations a difference this slight still reaches significance, which is precisely the situation where reporting a p-value alone would mislead a reader. A quarter of a CVSS point cannot separate the two groups in practice. This is a statistically detectable difference of no operational use, and it is the first piece of evidence that severity scoring is not measuring what defenders need it to measure.

Test 2: is ransomware adoption independent of SSVC Automatable?

H_0: the two variables are independent. H_1: they are associated. $\alpha = 0.05$.

The minimum expected cell count is 120, comfortably satisfying the chi-square assumption. The result is $\chi^2 = 16.29$ on 1 degree of freedom with $p = 5.4 \times 10^{-5}$, so H_0 is rejected. Vulnerabilities flagged automatable are linked to ransomware 25.7% of the time against 17.5% otherwise, an odds ratio of 1.64.

Cramer's V of 0.098 again indicates a weak association, so this is not a decisive predictor either. The direction is what makes it interesting. Automatability tracks ransomware adoption better than severity does, which supports the operational reading of Figure 4 and suggests that SSVC's decision points capture something about attacker economics that CVSS does not.

Test 3: does exploitation speed differ by attack vector?

H_0: median lag is equal across attack vectors. H_1: at least one differs. $\alpha = 0.05$.

This test was run on the 817 post-launch records only, because Figure 3 showed the full sample confounds attack vector with CVE age. Shapiro-Wilk rejects normality and Levene rejects equal variances, so Kruskal-Wallis was used in place of one-way ANOVA.

$H = 5.533$ with $p = 0.0629$. The p-value exceeds 0.05, so H_0 is not rejected. There is insufficient evidence that exploitation speed differs by attack vector.

Two points about reading this correctly. Failing to reject is not proof that the medians are equal; with 817 records split unevenly across three categories, and ADJACENT_NETWORK holding very few, the test has limited power. The result also sits just above the threshold, so it should be reported as

inconclusive rather than as a demonstration of no effect. Epsilon squared of 0.0065 indicates attack vector would account for well under 1% of the variation in lag even if the difference were real.

Running the same test on all 1,694 records would have returned a significant result, because attack vector correlates with CVE age and CVE age mechanically determines lag. That would have been a false positive produced entirely by the artefact in Figure 3, and it is a concrete illustration of why the restriction matters.

3.3 Model evaluation results

Part A: predicting exploitation lag

Table 3. Regression on log exploitation lag, cross-validated

Model	Subset	n	Test R^2	CV R^2	CV sd
Linear Regression	All records	1,694	0.4613	0.4671	0.045
Random Forest	All records	1,694	0.6310	0.5838	0.060
Gradient Boosting	All records	1,694	0.5979	0.5481	0.055
Linear Regression	Post-launch	817	-0.1513	-0.1718	0.141
Random Forest	Post-launch	817	-0.3560	-0.1607	0.178
Gradient Boosting	Post-launch	817	-0.3755	-0.1651	0.168

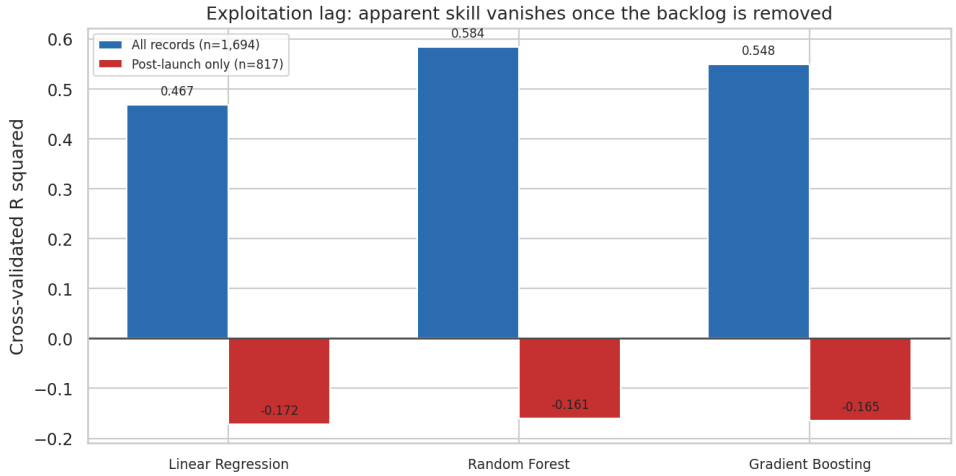


Figure 7. Apparent predictive skill disappears once the backlog is removed.

On all 1,694 records the random forest reaches a cross-validated R squared of 0.584 and looks like a competent model. On the 817 records where the question is actually well posed, every model falls below zero, meaning none of them beats simply predicting the mean lag for every vulnerability.

This is not a modelling failure to be tuned away, and no amount of hyperparameter search will fix it. The first set of numbers measures how well a model can recover the date the KEV catalog was created. The second measures whether a vulnerability's technical characteristics predict how fast attackers will weaponise it, and the answer is that they do not.

The result reproduces the finding that motivated FIRST.org to build EPSS as a scoring system separate from CVSS. Exploitation timing depends on attacker economics, target install base, toolkit availability and opportunism, none of which are recorded anywhere in a CVE record. A practitioner who tried to prioritise patching by predicting exploitation speed from CVE metadata would be working from noise.

Part B: predicting ransomware adoption

Table 4. Classification of ransomware adoption on the 339-row test set

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	0.788	0.462	0.171	0.250	0.724	0.406

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Gradient Boosting	0.788	0.450	0.129	0.200	0.695	0.374
Logistic Regression	0.578	0.268	0.600	0.370	0.644	0.343

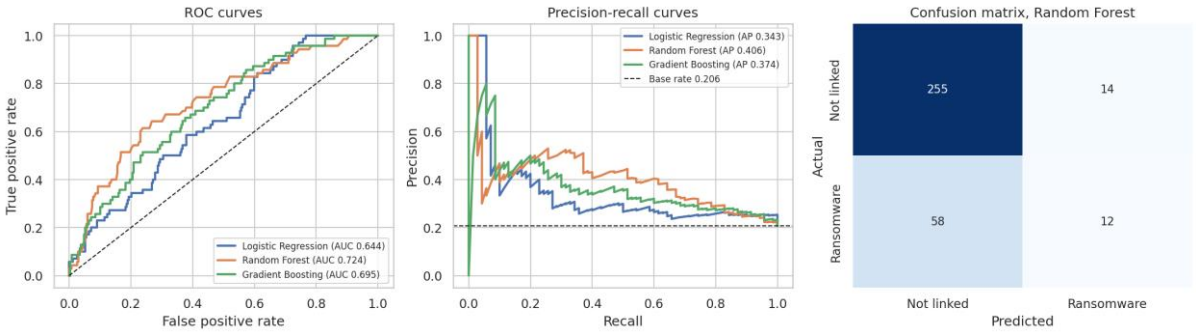


Figure 8. ROC curves, precision-recall curves and the confusion matrix.

The three models trade off differently, and which one is 'best' depends on the metric. The random forest has the strongest ranking ability at ROC-AUC 0.724, but at the default 0.5 threshold it recovers only 17.1% of ransomware-linked vulnerabilities. Logistic regression inverts this, catching 60.0% of them at a precision of 26.8%. Neither is wrong; they sit at different points on the same curve. In a triage setting the recall-oriented model is usually more useful, because missing an exploited vulnerability costs more than investigating one that turns out to be benign.

PR-AUC is the more honest headline for imbalanced data. The random forest reaches 0.406 against a base rate of 0.208, so it roughly doubles precision over random selection. That is a real improvement and a modest one. Accuracy of 78.8% looks respectable but is slightly below the 79.4% that predicting 'not ransomware' for everything would achieve on the same test set, which is why it is reported alongside the other metrics rather than on its own.

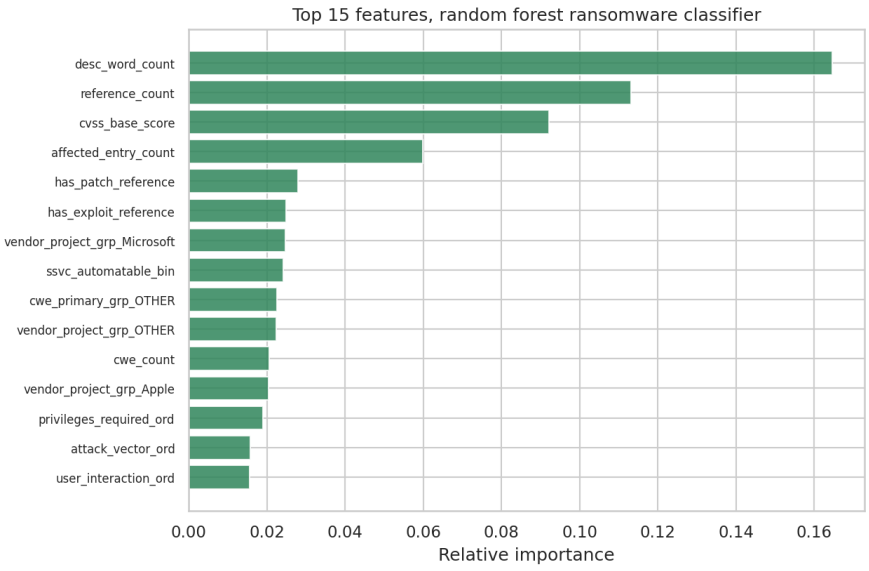


Figure 9. Top 15 features in the random forest classifier.

Description length and reference count rank highest, ahead of the CVSS base score. This should not be read as a causal claim. Both are continuous variables with many distinct values, which gives decision trees more places to split than a three-level ordinal feature offers, so impurity-based

importance flatters them. The defensible reading is narrower: severity is not the dominant signal, which agrees with Test 1 and with Figure 4.

4. Conclusion

Two results, one negative and one positive, and the negative one is more useful.

Exploitation speed cannot be predicted from a vulnerability's technical characteristics. A random forest trained on all 1,694 records reaches a cross-validated R squared of 0.584, which collapses to -0.161 once the analysis is restricted to the 817 CVEs published after the KEV catalog existed. The apparent skill was the model recovering a calendar boundary: the correlation between CVE publication year and exploitation lag is -0.916 across the full sample, because a vulnerability from 2005 could not possibly have been listed before the catalog opened in November 2021.

Ransomware adoption is weakly predictable. The random forest reaches ROC-AUC 0.724 and PR-AUC 0.406 against a 20.8% base rate, roughly doubling precision over random selection. The hypothesis tests point at why it is only weak: CVSS separates the groups by 0.275 points with a Cohen's d of 0.206, and SSVC Automatable gives an odds ratio of 1.64 with a Cramer's V of 0.098. Both are statistically significant and neither is operationally decisive.

Taken together these say something specific about vulnerability management. CVSS measures how bad a vulnerability could be in principle. It does not measure whether attackers will use it, and it does not measure how soon. Automatability, which is about whether exploitation can be scripted at volume, tracks ransomware adoption better than severity does, which fits how ransomware crews actually operate. This is the reasoning behind CISA's move to SSVC and behind EPSS existing as a separate system, and this analysis reaches it independently from public data.

4.1 Limitations

The catalog records detection, not exploitation. A vulnerability enters KEV when CISA has reliable evidence; vulnerabilities exploited quietly, or against organisations without the means to detect and report intrusions, never appear. The dataset measures what the US federal reporting ecosystem observed.

Every record is a positive case. All 1,694 rows are already confirmed exploited, so nothing here addresses the more valuable question of which of the roughly 250,000 published CVEs will ever be exploited. That needs the unexploited majority as a comparison group.

The ransomware label is one-sided. The field distinguishes 'Known' from 'Unknown', not from 'confirmed not used', so an unknown label may mean the link exists but is undocumented. The 20.8% positive rate is a floor rather than a measurement, and the classifier is partly learning which vulnerabilities get investigated and written up.

The lag analysis remains right-censored even after restriction. A CVE published in mid-2026 has had less calendar time in which to be listed than one published in 2022, which compresses observed lag for the newest records and is why the year-lag correlation settles at -0.29 rather than zero.

Vendor counts confound install base with security quality. Microsoft's 22.8% share reflects how widely its software is deployed and how closely it is monitored.

Some features are measured after the fact. Description length, reference count and the CVSS and SSVC fields were read from CVE records as they stood on 3 September 2026, not as they stood when each vulnerability was listed or linked to ransomware. References in particular accumulate as advisories and incident reports are published, so part of the signal behind the two highest-ranked classifier features may be information a defender would not yet have had. Point-in-time snapshots of each record would remove this.

4.2 Recommendations

Survival analysis is the correct tool for the lag question. Exploitation lag is a duration with censored observations, and a Cox proportional hazards model would handle the censoring that defeats ordinary regression here. This is the single most useful extension to the work.

Adding a negative class would transform the second analysis. Sampling CVEs that are not in KEV and modelling exploitation itself, rather than ransomware adoption among the already-exploited, would address the question defenders actually face. EPSS data could serve as an external benchmark.

For practice, the finding worth acting on is that severity ranking is a poor proxy for urgency. Organisations prioritising by CVSS alone are ordering their work by a measure that this data shows is nearly unrelated to whether a vulnerability is exploited quickly, and only weakly related to whether it becomes a ransomware vector. Checking membership of the KEV catalog directly, which is free and updated continuously, is more informative than any score computed here.

5. References

Cybersecurity and Infrastructure Security Agency (CISA). Known Exploited Vulnerabilities Catalog. Available at: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

Cybersecurity and Infrastructure Security Agency (CISA). cisagov/kev-data. GitHub. Available at: https://github.com/cisagov/kev-data

MITRE Corporation. CVE Program. Available at: https://www.cve.org

CVE Program. cvelistV5: CVE List V5 records. GitHub. Available at: https://github.com/CVEProject/cvelistV5

FIRST.org. Common Vulnerability Scoring System v3.1: Specification Document. Available at: https://www.first.org/cvss/v3.1/specification-document

FIRST.org. Exploit Prediction Scoring System (EPSS). Available at: https://www.first.org/epss/

Spring, J., Hatleback, E., Householder, A., Manion, A. and Shick, D. (2021). Prioritizing Vulnerability Response: A Stakeholder-Specific Vulnerability Categorization (Version 2.0). Carnegie Mellon University, Software Engineering Institute.

MITRE Corporation. Common Weakness Enumeration (CWE). Available at: https://cwe.mitre.org

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56-61.

Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585, 357-362.

Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods, 17, 261-272.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.

Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), 1189-1232.

Welch, B. L. (1947). The generalization of Student's problem when several different population variances are involved. Biometrika, 34(1-2), 28-35.

Kruskal, W. H. and Wallis, W. A. (1952). Use of Ranks in One-Criterion Variance Analysis. Journal of the American Statistical Association, 47(260), 583-621.

Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences. 2nd ed. Lawrence Erlbaum Associates.

Saito, T. and Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, 10(3), e0118432.

6. Repository

The build script, dataset, executed notebook and this report are available at the link below. The notebook runs top to bottom without modification and, with the package versions pinned in requirements.txt, reproduces every figure and number quoted here. Other scikit-learn versions shift the tree-based results: ROC-AUC and PR-AUC move only in the third decimal place, but the random forest's recall at the 0.5 threshold can change substantially, which is a further reason to read the

ranking metrics rather than the threshold ones. Because the build script fetches live data, re-running it will produce a slightly larger dataset than the one analysed; the collected CSV is included so the analysis runs on the same snapshot.

Repository URL .....

Files: KEV_Ransomware_Analysis.ipynb, build_dataset.py, data/kev_cve_enriched_raw.csv, data/cisa_kev_catalog.csv, data/kev_cve_cleaned.csv, figures/, results_summary.json, requirements.txt, README.md